How Portfolio Workbench works

Deep analysis generated 2026-09-07 · app booted via npm run dev; screenshots are real captures · model claude-sonnet-5

Overview

Portfolio Workbench is Avishek Chandra's personal portfolio system, built to document side projects as plain Markdown/YAML files under Git version control rather than in a database or hosted CMS. It presents itself to the public as "Avishek's Portfolio" — a browsable index of project cards with a carousel, search/filter, and per-project detail pages — while doubling as a full authoring tool for its owner. The core idea, spelled out in the README, is that "Markdown is the record, Git is the history, the interface is the view": every project is a folder (`src/content/projects/<id>-<slug>/`) with an `index.md` (Zod-validated front matter) and an `artifacts/` directory of images, PDFs, and videos, and Astro statically renders those folders into pages at build time.

The same codebase runs in two distinct modes. In **local workbench** mode (`npm run dev`), the app runs on `localhost:4321` with a development-only Vite middleware (`src/lib/local-editor.ts`) that writes directly to the filesystem; editing private projects requires a WebAuthn "device unlock" (Windows Hello, fingerprint, or PIN) implemented in `src/lib/device-auth.ts` and `src/lib/device-unlock-client.ts`. In **hosted** mode (`PUBLIC_ONLY=1`, deployed to GitHub Pages as an installable PWA), there is no server at all — every edit becomes a commit to GitHub via the Contents API, authenticated with a fine-grained personal access token the owner pastes once into a "Connections" dialog in the header. Deletions are soft (a `deleted: true` flag in front matter), so history is preserved in Git even though the live site stops rendering the project.

The audience is effectively just the owner (as author/editor) and site visitors (as read-only viewers of public projects); there's no multi-user or account system beyond the single owner's GitHub token and optional AI API key. Visitors see a filterable "project reel" home page and can drill into any public project's detail page, which shows its description, why-it-was-built narrative, status/privacy/date metadata, and an artifact carousel. Private projects render as locked stub cards on the public build; the real content lives in a separate `workbench-private` GitHub repository and is only fetched client-side after the owner authenticates with GitHub in the browser (`PrivateVault.astro`).

A distinguishing feature is AI-assisted authoring: an owner can paste a repository (and optionally pull-request) URL, and a "Draft from repository" flow gathers redacted, budgeted evidence from the GitHub API (`src/lib/ai/evidence.ts`), drafts the project's description and why-it-was-built text via Anthropic (directly) or OpenAI (via a bundled Cloudflare Worker CORS proxy), and then queues a background "deep analysis" GitHub Actions job. That job (`scripts/deep-analysis.mjs`) clones the actual linked repository, boots it (Docker Compose/Dockerfile/npm/python heuristics), crawls it with a real headless browser, screenshots every page, documents every control with the model, typesets a PDF report, and commits everything back into the project's artifacts — continuing to run even if the browser tab or device is closed, with the site polling a request-marker file to show live progress.

Architecture

The stack is Astro (static output, TypeScript) with content collections backed by Zod schemas (`src/lib/project-schema.ts`) that validate every project's front matter at build time. Pages are statically generated per project via `getStaticPaths` in `src/pages/projects/[slug].astro`, and a stream of small library modules under `src/lib/` supply shared logic: `projects.ts` (loading/sorting/filtering projects and artifacts, base-path prefixing), `artifact-carousel.ts` and `artifact-types.ts` (rendering/typing images, videos, PDFs, and generic files), `frontmatter.ts` (YAML parse/serialize), `github-client.ts` (browser-side GitHub Contents API wrapper with token storage and a parallel "vault" API for the private repo), `project-editor.ts` and `field-editor.ts` (shared modal-based inline field editing used by both the public hosted editor and the private vault), and the `ai/` subdirectory (evidence collection, model provider abstraction, draft orchestration, and analysis-request polling).

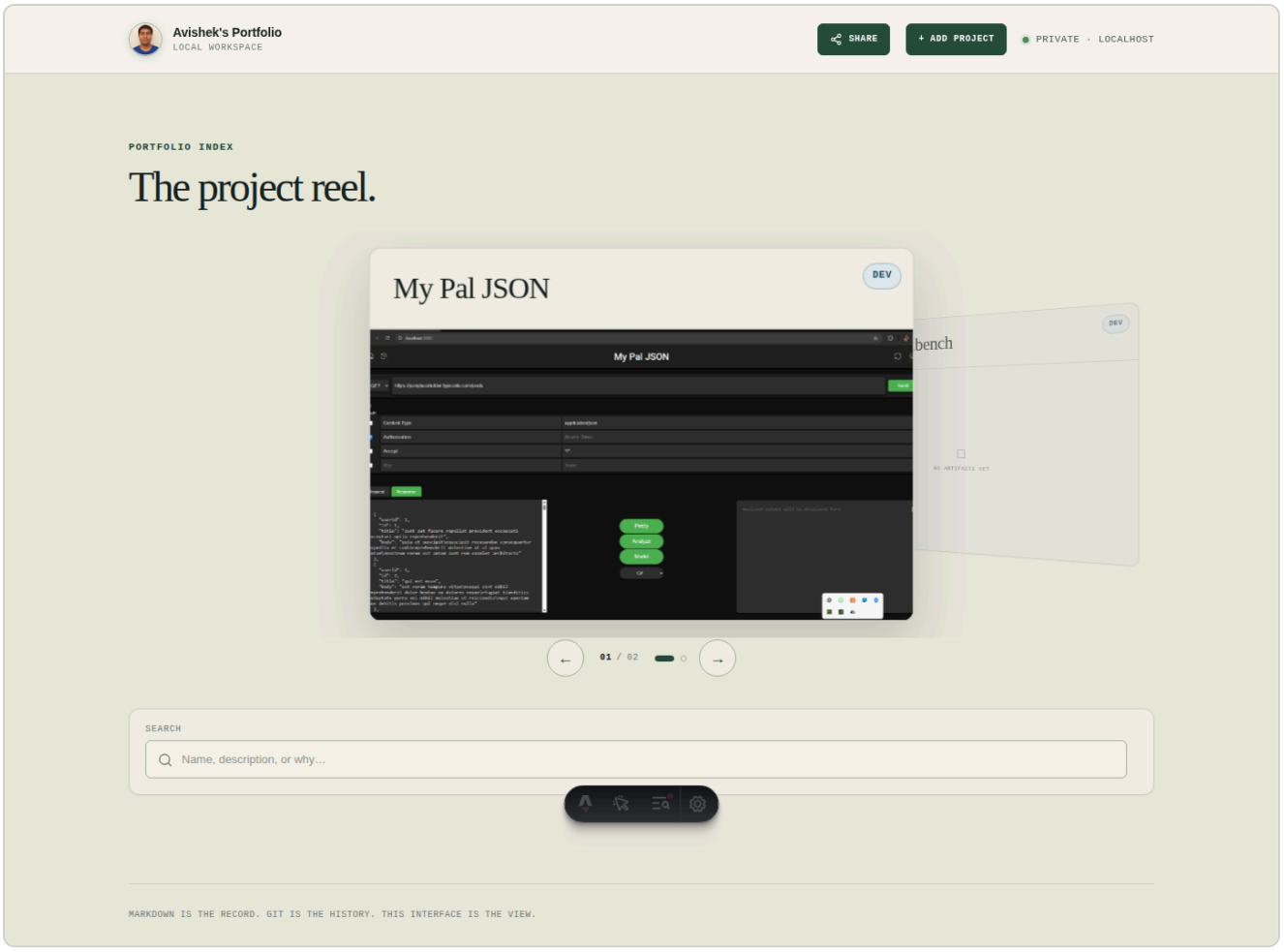
Two editing backends implement a common `EditorBackend` interface: `RemoteEditor.astro` (commits to the public `project-workbench` repo) and `PrivateVault.astro` (commits to the separate `workbench-private` repo, fetched and

unlocked client-side after GitHub auth). `RemoteCreate.astro` handles hosted-site project creation (always public), while local-mode creation and editing go through `manage.astro`'s form plus the local-editor Vite middleware (`src/lib/local-editor.ts`), which is bound to the loopback interface and gated by WebAuthn device unlock (`device-auth.ts`). `ProjectDelete.astro` implements the soft-delete pattern shared by both public and private records.

Data flow for a page view: Astro's `getCollection('projects')` loads and validates all `index.md` files via `glob` loader + `projectMetadataSchema`; `aliveProjects`/`listedProjects` in `projects.ts` filter out soft-deleted and (in public builds) private/unpublished records; `loadArtifacts` reads each project's `artifacts/` folder from disk and produces ordered, typed artifact descriptors served through the dynamic route `src/pages/project-artifacts/[project]/[...file].ts`. For hosted editing, every write is a two-step "read current file+sha, mutate front matter, PUT back" operation via `makeFrontmatterPatcher`, with a retry on GitHub's 409 conflict. Background automation is wired through three GitHub Actions workflows: `deploy.yml` (build+publish on every push to main, generating private stubs first via `generate-private-stubs.mjs`), `deep-analysis.yml` (triggered by commits to `analysis-requests/**`, running `scripts/deep-analysis.mjs`), and `convert-pptx.yml` (converts uploaded PowerPoint artifacts to PDF via LibreOffice). AI calls are constrained by an owner-allowlist (`ALLOWED_OWNERS = ['aviman1258']` in `ai-complete.ts`) and secret redaction in `ai/evidence.ts`, so the model only ever sees sanitized data from the owner's own repos.

Page: /

The public home page — a browsable, searchable index of all live (non-deleted, and in hosted builds, public-only) projects, presented as a "project reel" carousel with cards.



Walkthrough

A visitor lands on the home page and sees the header (brand/contact button, Share, "+ Add project", and either a GitHub connections icon in hosted mode or a "Private · localhost" indicator in local mode) followed by "The project reel." heading and a horizontally-paged coverflow carousel of project cards, each showing a status badge, name, and featured artifact (or an empty-state "No artifacts yet" placeholder, or a locked "Unlock private project" tile for private records). Below the carousel is a position indicator, dot navigation per project, and left/right arrow buttons. A search field lets the visitor filter cards live by name, description, or why-text (matched against a precomputed `data-search` string built server-side in `ProjectCard.astro`); a "Clear filters" button and an "No matching projects" empty state appear as needed. If a private project card is present and unlocked locally, a "Private project" panel offers a "Unlock with Windows Hello" button that runs the WebAuthn device-unlock ceremony (`src/lib/device-unlock-client.ts`) before revealing it. Clicking a card's artifact or name navigates to that project's detail page at `/projects/<slug>`.

Controls

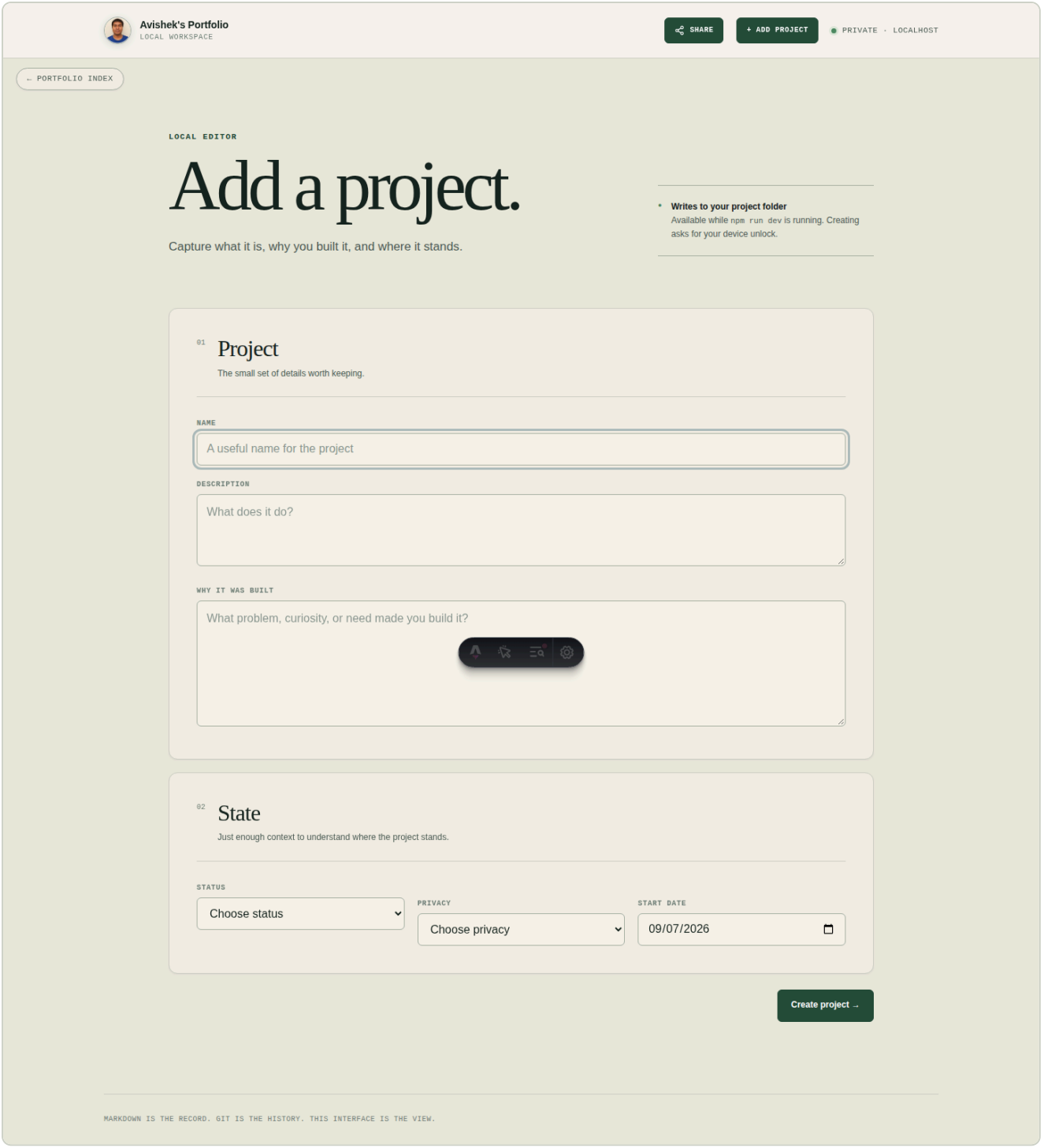
CONTROL	WHAT IT DOES
Contact card for Avishek	opens a dialog showing the owner's business card (name, email, phone, certification badges) — pure client-side dialog toggle, no backend call.

Chandra (button, `data-card-open` in BaseLayout.astro)	
Share (button, `data-share-open`)	opens a share dialog (native share sheet / copy link) defined in BaseLayout.astro's inline script (not shown in full, but wired via `data-share-open`).
+ Add project (a, links to `withBase('/manage')`)	navigates to the `/manage` page to create a new project; styled with a `needs-github` hint in hosted builds if GitHub isn't connected.
My Pal JSON / Portfolio Workbench (a, project card links)	navigate to each project's detail page at `/projects/<slug>` (or `/projects/<slug>#edit` when clicking a featured artifact), rendered by `src/pages/projects/[slug].astro`.
□ No artifacts yet (a, `project-card__empty`)	for projects lacking artifacts, links straight to the project detail page.
Show previous project / Show next project (buttons, `data-project-coverflow-previous` / `-next`)	step the coverflow's active-card index backward/forward via the inline script's `showCoverflowCard` function, updating position text and ARIA state.
Show My Pal JSON / Show Portfolio Workbench (buttons, `data-project-coverflow-dot`)	jump the coverflow directly to that project's card by index, calling `showCoverflowCard(card)`.
Search input (`data-filter-search`, type=search)	filters visible project cards client-side by matching the query against each card's precomputed `data-search` string; toggles the empty-state and "Clear filters" button.

Unlock with Windows Hello (button, `data-private-unlock-button`)	triggers `deviceUnlock()` from `device-unlock-client.ts`, running the WebAuthn registration/authentication ceremony against `/api/local/device-auth/*` endpoints (local-editor backend) to reveal a private project's card.

Page: /manage

The "Add a project" page — creates a new project record, either by committing to GitHub (hosted build, via `RemoteCreate.astro`) or by writing directly to the local filesystem behind a device-unlock gate (local dev build, via the inline form in `manage.astro`).



Walkthrough

In the hosted build, the visitor (presumably the owner, connected via GitHub token) sees the `RemoteCreate` component: a "Repository link" field with a "✦ Draft from repository" button, an optional "Pull request link,"

Name/Description/Why textareas, Status and Start date fields, and a "Create project →" submit button; if no GitHub token is stored, a hint asks them to connect via the header's connections icon instead of showing the form. In local dev, the page instead shows a full multi-section form ("01 Project" — name/description/why; "02 State" — status, privacy, start date) with a "Writes to your project folder" note explaining that submission requires a device unlock. On submit, the local script calls `ensureDeviceUnlock()` (WebAuthn ceremony) before posting to the local-editor backend, and inline validation errors are shown per-field (`showFormError`/`clearFieldError`) if the backend (`src/lib/local-editor.ts`'s `createProject`, validated against `projectDraftSchema`) rejects the submission (e.g., duplicate slug).

Controls

CONTROL	WHAT IT DOES
Contact card for Avishek Chandra (button)	same business-card dialog as the home page (BaseLayout.astro).
Share (button)	same share dialog as the home page.
+ Add project (a)	self-referential link back to `/manage` (present in header on every page).
Name input	project name field; used to derive the kebab-case slug via `slugify()` in `frontmatter.ts`/`local-editor.ts`.
Description textarea	"What does it do?" — maps to the `description` front-matter field, required (max 500 chars per `projectDraftSchema`).
Why textarea	"What problem, curiosity, or need made you build it?" — maps to `why`, required (max 3000 chars).
Status select (Choose status: idea/dev/review/delivered)	sets the project's `status` front-matter field, validated against `projectStatuses` in `project-schema.ts`.
Privacy select (Choose privacy: private/public)	sets `privacy`; in local mode `private` triggers the WebAuthn device-unlock requirement on future edits.
Start date input	sets `startDate` (ISO date), defaults to today.
Create project → (button, submit)	local mode — calls `ensureDeviceUnlock()` then posts the draft to the local-editor Vite middleware's create-project handler (`createProject` in `local-editor.ts`), which validates via `projectDraftSchema`, checks slug uniqueness, and writes a new `index.md` + `artifacts/` folder; hosted mode — `RemoteCreate.astro`'s submit handler commits a new project folder to GitHub via `putFile`, following the duplicate-repository guard dialog logic.
✦ Draft from	enabled once a GitHub-style repo URL is detected in the Repository link field (`looksLikeRepo()`); calls `draftFromRepository()` (`ai/repo-draft.ts`) to fetch evidence and draft the

repository (button, `data-create-ai`, hosted only)	name/description/why via the configured AI provider.

Page: /projects/my-pal-json

Public detail page for the "My Pal JSON" project, showing its description, why-it-was-built narrative, metadata, and artifact carousel, with inline editing enabled for the hosted-site owner.

PROJECT / 004 DEV

My Pal JSON

My Pal JSON is a web-based tool for testing APIs and analyzing JSON responses with features like dynamic header management, request/response viewing, and JSON structure visualization. It can generate code models in six programming languages (C#, Python, JavaScript, C++, Java, Go) and includes dark/light theme switching.

PRIVACY
Public
Ready to share.

STATUS	STARTED	UPDATED	REPOSITORY	PULL REQUEST
Dev	Oct 3, 2024	Sep 7, 2026	github.com/aviman1258/my-pal-json	—

LOCAL EDITOR

Manage this project

Changes are saved directly to this project's Markdown record.

✦ Edit project details

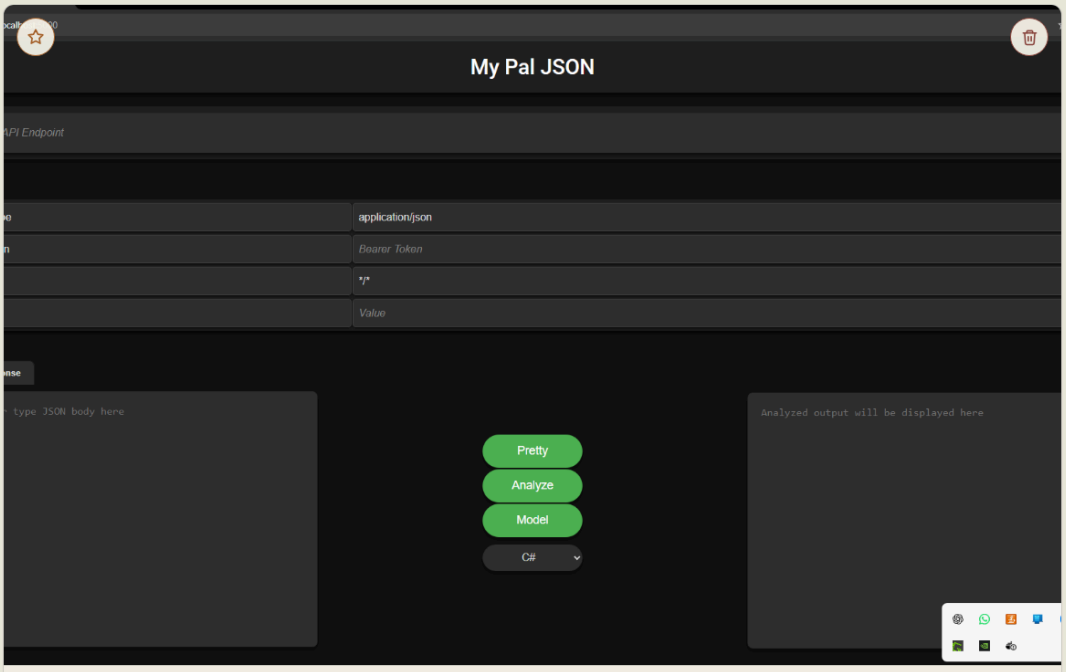
Why it was built

I built this project to eliminate the friction of context-switching betwe[illegible]orking with APIs—I found myself constantly moving JSON between my request client, a formatter, a structure inspector, and code generators. My Pal JSON consolidates all of these workflows into a single lightweight browser workspace, so I can send API requests, analyze responses, explore JSON structure, and generate code models without leaving the application. This unified approach saves time and keeps my focus on the actual API exploration rather than tool management.

Evidence from the work

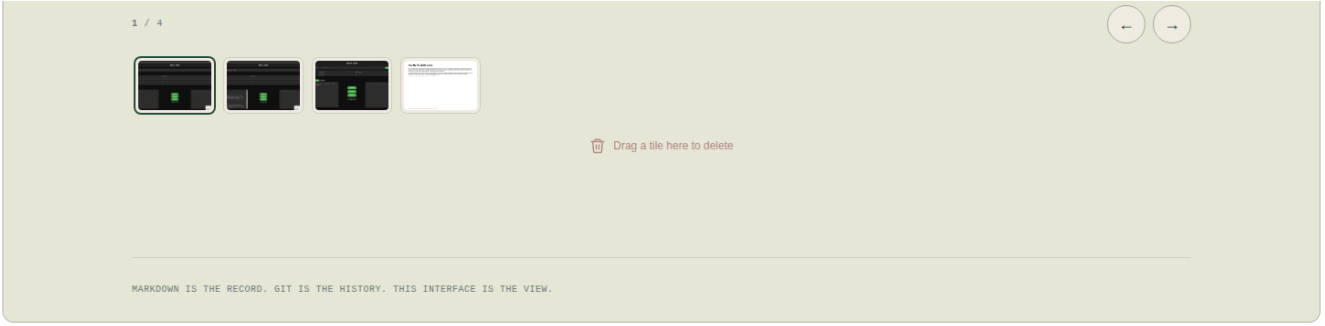
Images, documents, and recordings stored alongside this project. Drop files onto the carousel to add them, drag the tiles to reorder.

+ Add media



mypaljson-20260902055709.png
IMAGE

OPEN ORIGINAL ↗



Walkthrough

The visitor sees the project hero (ID badge "004", status badge "dev", title "My Pal JSON", description) followed by a metadata strip (Status, Started, Updated, Repository link to `github.com/aviman1258/my-pal-json`, Pull request). Because this is a public, non-private project on the hosted build, `RemoteEditor.astro` is active, adding pencil icons next to editable fields for the owner once connected. An expandable `<details>` "✦ Edit project details" panel contains a full inline edit form (name, description, why, status, privacy, start date selects/inputs) with a "Save changes" submit button. Below that, an "+ Add media" button opens a file picker to upload new artifacts, and the artifact carousel shows four items — three images and a PDF — each with an "Open" link, a "Feature" star toggle (or "is the featured artifact" indicator for the current hero image), and a "Delete" trash button, plus carousel navigation (previous/next, thumbnail-select buttons) and an "Open original ↗" link per slide.

Controls

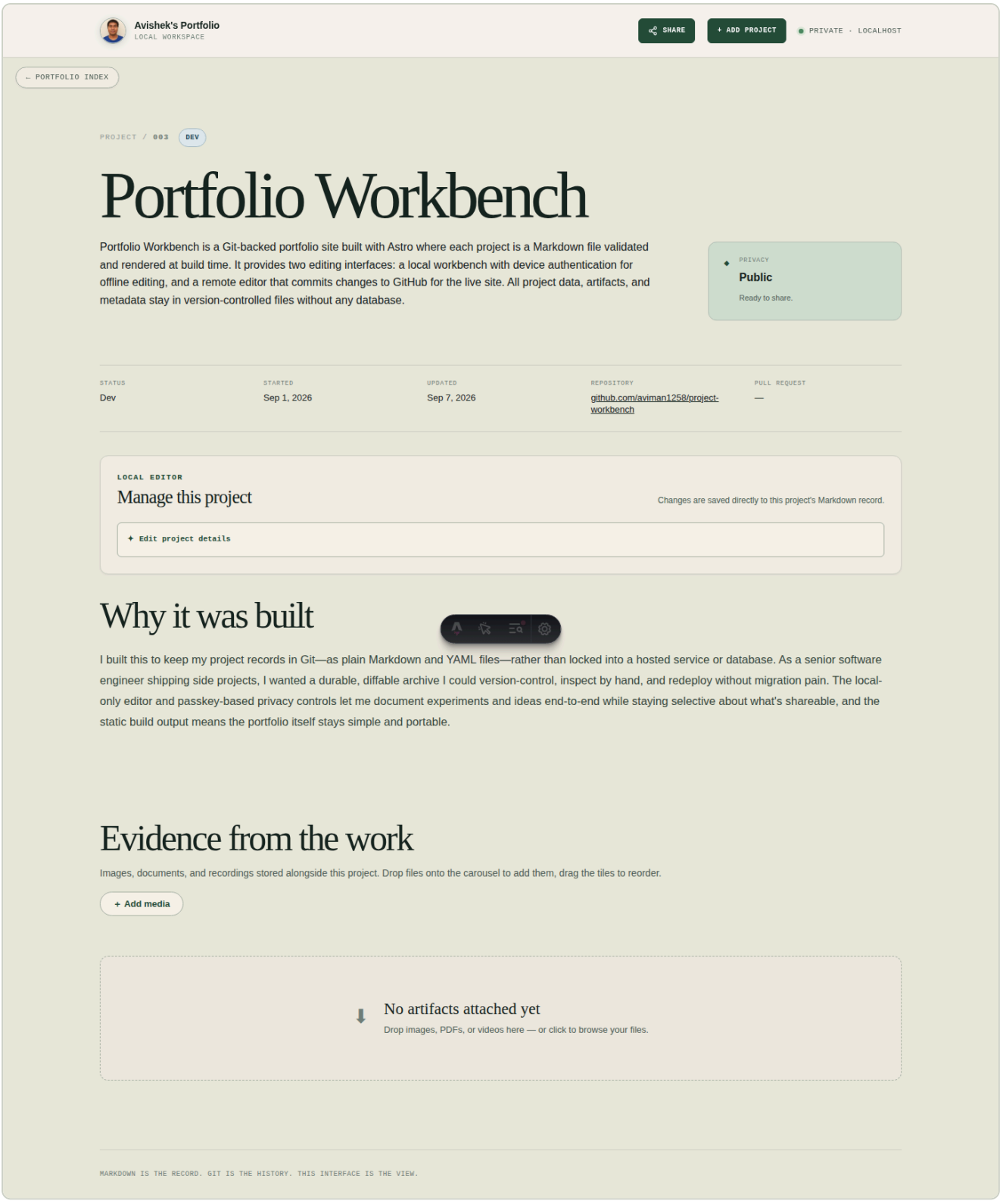
CONTROL	WHAT IT DOES
Contact card for Avishek Chandra / Share / + Add project (buttons/link)	same global header controls as other pages (BaseLayout.astro).
github.com/aviman1258/my-pal-json (a)	external link to the linked GitHub repository, opened in a new tab (`target="_blank"`).
✦ Edit project details (summary, `<details>`)	expands/collapses the inline editing form for name/description/why/status/privacy/startDate.
name / description / why textareas, status/privacy selects, startDate input (within the	bound to the project's front-matter fields; on "Save changes" they patch `index.md` via `makeFrontmatterPatcher`/`RemoteEditor`'s `EditorBackend.putIndex`, committing to `src/content/projects/004-my-pal-json/index.md` on GitHub.

edit-details form)	
Save changes (button, submit)	commits the edited front matter fields to GitHub via `patchFrontmatter`, using `getFile`/`putFile` from `github-client.ts`, with `onSaved` showing "Saved. The site rebuilds in about a minute."
+ Add media (button, `data-remote-media-open`)	opens the hidden file input (`data-remote-media-input`), letting the owner pick media files (≤25MB, must have an extension) that get committed as new artifacts via `putArtifact`/`putFile` to `src/content/projects/004-my-pal-json/artifacts/`.
Open <filename> (a, per artifact slide)	opens the artifact file directly (served by `src/pages/project-artifacts/[project]/[...file].ts`) in a new tab.
Feature <filename> / <filename> is the featured artifact (button, `data-media-feature-trigger`)	sets/toggles the project's `featuredArtifact` front-matter field to that filename, updating which artifact is the card's hero image; wired through `project-editor.ts`'s carousel feature handling.
Delete <filename> (button, `data-media-delete-trigger`)	opens a confirm dialog then deletes the artifact file from the GitHub repo via `deleteFile`.
Show previous artifact / Show next artifact (buttons, `data-carousel-previous`/`-next`)	step the carousel's active slide index, from `artifact-carousel.ts`'s `wireCarousel`.
Show <filename> (buttons,	jump the carousel directly to that artifact's slide.

`data-carousel-thumbnail`)

Page: /projects/porfolio-workbench

Public detail page for the "Portfolio Workbench" project itself (this application), showing its own description, rationale, metadata, and (currently empty) artifact area, with the same hosted inline-editing controls as any other public project.



Walkthrough

Identical structural pattern to the My Pal JSON page: hero with ID "003", status badge "dev", title "Portfolio Workbench", description text describing the Git-backed Astro portfolio site itself; metadata strip with a link to `github.com/aviman1258/project-workbench`; an expandable "✦ Edit project details" panel with the same field-editing form and "Save changes" button; and an "+ Add media" button. Because this project currently has no artifacts, the carousel area instead shows an empty-state message: "⬇ No artifacts attached yet — Drop images, PDFs, or videos here — or click to browse your files," reflecting `carouselHtml()`'s empty branch in `artifact-carousel.ts` and doubling as a drop zone when in local/editing mode.

Controls

CONTROL	WHAT IT DOES
Contact card for Avishek Chandra / Share / + Add project (buttons/link)	same global header controls (BaseLayout.astro).
github.com/aviman1258/project-workbench (a)	external link to this project's own GitHub repository.
✦ Edit project details (summary)	expands the inline edit form for this project's front matter, same as My Pal JSON.
name / description / why / status / privacy / startDate fields	same behavior as My Pal JSON — edits patch `src/content/projects/003-porfolio-workbench/index.md` via GitHub commit.
Save changes (button, submit)	commits field edits via `RemoteEditor`'s `EditorBackend`, same flow as other public projects.
+ Add media (button)	opens the hidden file input to upload the project's first artifacts, committing new files to `src/content/projects/003-porfolio-workbench/artifacts/` via `putArtifact`/`putFile`.
⬇ No artifacts attached yet / drop zone (button/div, `data-media-dropzone`)	acts as both an empty-state message and (when `localEditing` is enabled) a drag-and-drop target for uploading artifact files directly onto the carousel area.